Exercise 1

meiermis

1. Use cases

1.1 Selected use cases

1. Keeping and enhancing Shorthaul PAX, regional Cargo and GA Keeping what we already have as core revenue and value creation driver shall stay primary focus and priority
2. UAM PAX Transport: eVTOL Aircraft for short range PAX transport (mostly airport shuttle and back) to the city center 70km away seems feasable
3. UAV Logistics: Could be applicable to our airport, mostly for incoming regional Cargo via Airplane that can then be switched to UAV for further delivery to a main cargo hub closer to the city center for delivery.

1.2 Required Infrastructure and users

- Vertiports for UAV/UAM
- Dedicated Drone Approach/Departure segments that do not interfere traditional OPS
- Electric Charging and Storage infrastructure
- eVTOL sharing companies could become new users

Points 2 and 3 regarding UAM/UAV are more in a 20-50 years horizon and therefore should be kept as options in a dynamic forecast. A high uncertainty regarding regulations and technology development is still present.

2. GBM Demand Scenarios

In [7]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

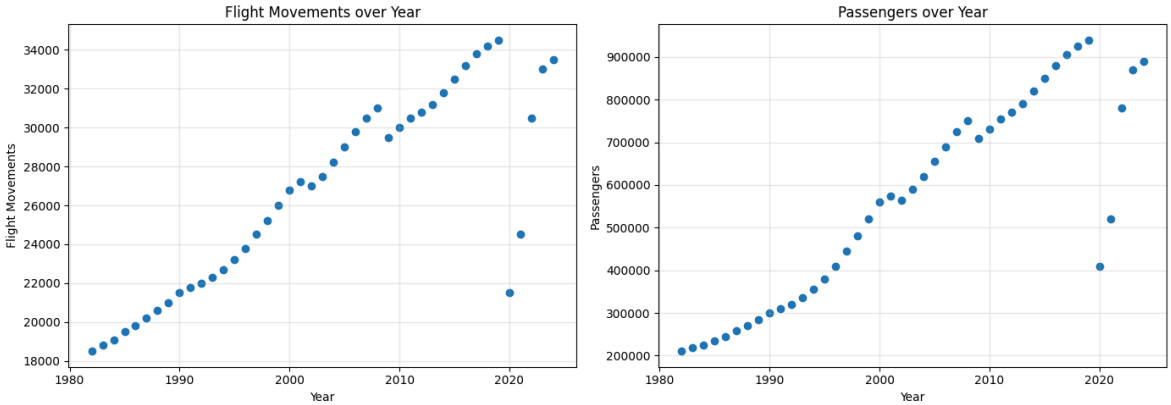
In [2]:
```
aerovale_demand = pd.read_csv("aerovale_demand.csv", sep=";")
aerovale_demand.head()
```

Out[2]:

	Year	Flight_Movements	Passengers
0	1982	18500	210000
1	1983	18800	218000
2	1984	19100	225000
3	1985	19500	235000
4	1986	19800	245000

In [25]:

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.scatter(aerovale_demand['Year'], aerovale_demand['Flight_Movements'])
ax1.set_xlabel('Year')
ax1.set_ylabel('Flight Movements')
ax1.set_title('Flight Movements over Year')
ax1.grid(True, alpha=0.3)

ax2.scatter(aerovale_demand['Year'], aerovale_demand['Passengers'])
ax2.set_xlabel('Year')
ax2.set_ylabel('Passengers')
ax2.set_title('Passengers over Year')
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



In [32]:

```
#remove the year 2020, 2021 and 2022, because of the covid-19 pandemic
aerovale_demand_cleaned = aerovale_demand[(aerovale_demand['Year'] != 202
```

In [31]:

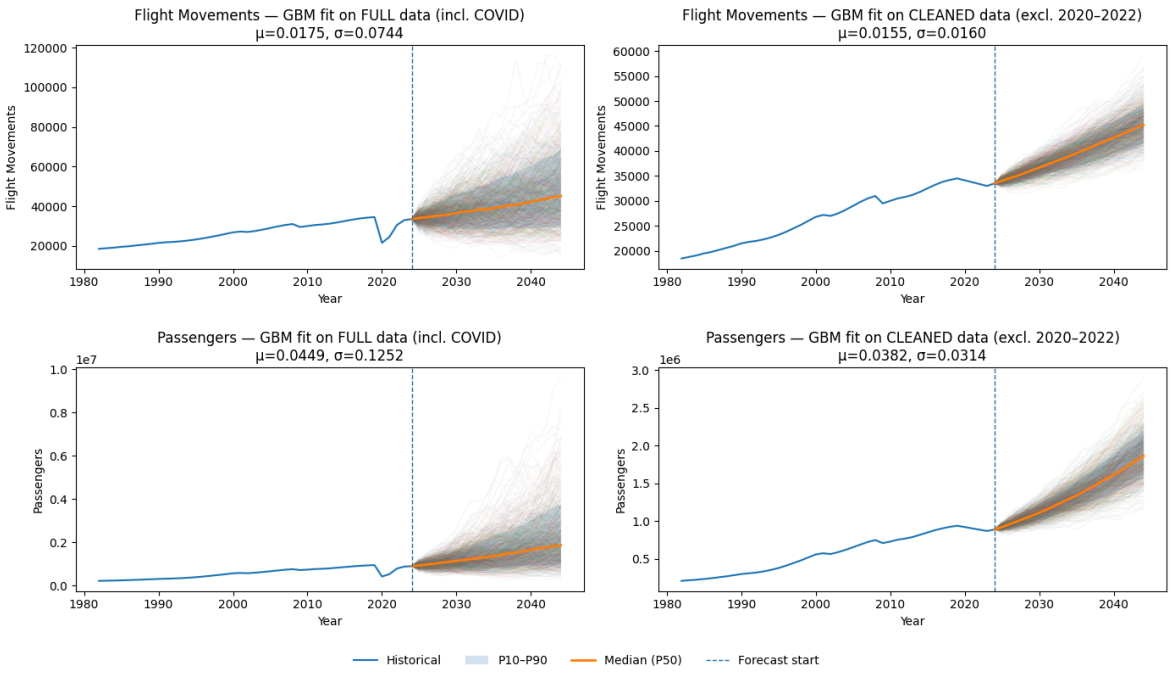
```
df = aerovale_demand_cleaned.copy().sort_values("Year")

g_f = df["Flight_Movements"].pct_change().dropna()   # relative annual gr
g_p = df["Passengers"].pct_change().dropna()

mu_f = g_f.mean()
sig_f = g_f.std(ddof=0)   # entspricht deiner Formel mit /T

mu_p = g_p.mean()
sig_p = g_p.std(ddof=0)

mu_f, sig_f, mu_p, sig_p
```

Out[31]:
```
(np.float64(0.017534667271089394),
 np.float64(0.07442923766670641),
 np.float64(0.044873743931640196),
 np.float64(0.12521694014903587))
```

In [33]:
```
# --- assume these exist ---
# aerovale_demand
# aerovale_demand_cleaned = aerovale_demand[(aerovale_demand['Year'] != 2

# -----------------------
# Settings
# -----------------------
years_future = 20
dt = 1.0
n_paths = 1000
seed = 42
clip_negative_multipliers = True  # set False if you want "pure" Euler (c

# -----------------------
# Helpers
# -----------------------
def fit_mu_sigma(df: pd.DataFrame, col: str):
    df = df.sort_values("Year")
    g = df[col].pct_change().dropna()              # relative annual grow
    mu = float(g.mean())
    sigma = float(g.std(ddof=0))                   # matches /T in your f
    return mu, sigma

def simulate_gbm_euler(D0: float, mu: float, sigma: float, *, years_futur
    eps = rng.normal(0.0, 1.0, size=(years_future, n_paths))
    factors = 1.0 + mu * dt + sigma * eps * np.sqrt(dt)  # your ΔD model
    if clip_mult:
        factors = np.clip(factors, 0.0, None)
    paths = np.vstack([np.full((1, n_paths), D0), D0 * np.cumprod(factors
    return paths  # shape (years_future+1, n_paths)

def summarize(paths: np.ndarray):
    q10 = np.quantile(paths, 0.10, axis=1)
    q50 = np.quantile(paths, 0.50, axis=1)
    q90 = np.quantile(paths, 0.90, axis=1)
    return q10, q50, q90

def plot_panel(ax, years_hist, y_hist, years_all, paths, title, ylabel, l
    q10, q50, q90 = summarize(paths)

    # historical
    ax.plot(years_hist, y_hist, label="Historical")

    # paths (light)
    ax.plot(years_all, paths, linewidth=0.6, alpha=0.05)

    # fan + median
    ax.fill_between(years_all, q10, q90, alpha=0.2, label="P10–P90")
    ax.plot(years_all, q50, linewidth=2.0, label="Median (P50)")

    ax.axvline(last_year, linestyle="--", linewidth=1, label="Forecast st
    ax.set_title(title)
    ax.set_xlabel("Year")
    ax.set_ylabel(ylabel)
```

```
# ------------------------
# Prepare datasets
# ------------------------
full = aerovale_demand.copy().sort_values("Year")
clean = aerovale_demand_cleaned.copy().sort_values("Year")

# pick start values consistently (use the last available year in EACH dat
D0_f_full = float(full["Flight_Movements"].iloc[-1])
D0_p_full = float(full["Passengers"].iloc[-1])
last_year_full = int(full["Year"].iloc[-1])

D0_f_clean = float(clean["Flight_Movements"].iloc[-1])
D0_p_clean = float(clean["Passengers"].iloc[-1])
last_year_clean = int(clean["Year"].iloc[-1])

# future year axes (can differ if last_year differs)
future_years_full = np.arange(last_year_full + 1, last_year_full + years_
years_all_full = np.concatenate([[last_year_full], future_years_full])

future_years_clean = np.arange(last_year_clean + 1, last_year_clean + yea
years_all_clean = np.concatenate([[last_year_clean], future_years_clean])

# ------------------------
# Fit + simulate (FULL vs CLEAN) for both series
# ------------------------
rng = np.random.default_rng(seed)

mu_f_full, sig_f_full = fit_mu_sigma(full, "Flight_Movements")
mu_p_full, sig_p_full = fit_mu_sigma(full, "Passengers")

mu_f_clean, sig_f_clean = fit_mu_sigma(clean, "Flight_Movements")
mu_p_clean, sig_p_clean = fit_mu_sigma(clean, "Passengers")

paths_f_full = simulate_gbm_euler(D0_f_full, mu_f_full, sig_f_full,
                                  years_future=years_future, n_paths=n_pat
                                  rng=rng, clip_mult=clip_negative_multipl
paths_p_full = simulate_gbm_euler(D0_p_full, mu_p_full, sig_p_full,
                                  years_future=years_future, n_paths=n_pat
                                  rng=rng, clip_mult=clip_negative_multipl

paths_f_clean = simulate_gbm_euler(D0_f_clean, mu_f_clean, sig_f_clean,
                                   years_future=years_future, n_paths=n_pa
                                   rng=rng, clip_mult=clip_negative_multip
paths_p_clean = simulate_gbm_euler(D0_p_clean, mu_p_clean, sig_p_clean,
                                   years_future=years_future, n_paths=n_pa
                                   rng=rng, clip_mult=clip_negative_multip

# ------------------------
# Plot in subplots (2x2)
# ------------------------
fig, axes = plt.subplots(2, 2, figsize=(14, 8), sharex=False)

plot_panel(
    axes[0, 0],
    years_hist=full["Year"].values,
    y_hist=full["Flight_Movements"].values,
    years_all=years_all_full,
    paths=paths_f_full,
    title=f"Flight Movements — GBM fit on FULL data (incl. COVID)\nμ={mu_
    ylabel="Flight Movements",
```

```
    last_year=last_year_full,
)

plot_panel(
    axes[0, 1],
    years_hist=clean["Year"].values,
    y_hist=clean["Flight_Movements"].values,
    years_all=years_all_clean,
    paths=paths_f_clean,
    title=f"Flight Movements — GBM fit on CLEANED data (excl. 2020–2022)\
    ylabel="Flight Movements",
    last_year=last_year_clean,
)

plot_panel(
    axes[1, 0],
    years_hist=full["Year"].values,
    y_hist=full["Passengers"].values,
    years_all=years_all_full,
    paths=paths_p_full,
    title=f"Passengers — GBM fit on FULL data (incl. COVID)\nμ={mu_p_full
    ylabel="Passengers",
    last_year=last_year_full,
)

plot_panel(
    axes[1, 1],
    years_hist=clean["Year"].values,
    y_hist=clean["Passengers"].values,
    years_all=years_all_clean,
    paths=paths_p_clean,
    title=f"Passengers — GBM fit on CLEANED data (excl. 2020–2022)\nμ={mu
    ylabel="Passengers",
    last_year=last_year_clean,
)

# one shared legend (avoid clutter)
handles, labels = axes[0, 0].get_legend_handles_labels()
fig.legend(handles, labels, loc="lower center", ncol=4, frameon=False)

plt.tight_layout(rect=(0, 0.05, 1, 1))
plt.show()
```



3. Scenarios for Market Share of UAV / eVTOL

Adoption of novel aviation technologies typically follows S-curve dynamics (slow introduction, rapid growth, saturation), consistent with diffusion models used in technology forecasting (logistic/Bass). Market entry timing, growth speed, and ultimate saturation are uncertain; sampling these parameters and running Monte Carlo produces a diverse scenario set rather than a single forecast.

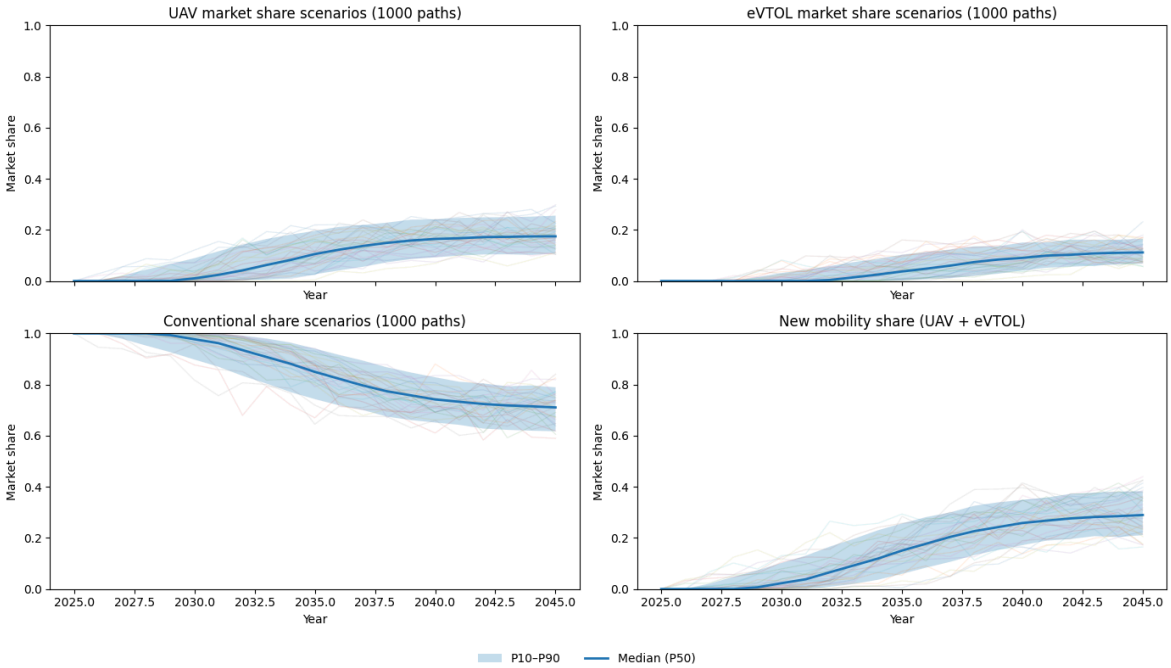
Technology Readiness Level (TRL) nach NotebookLM: Die folgenden TRL-Einschätzungen stammen nicht direkt aus den Quellen und dienen als externe Orientierungshilfe. UAV (Uncrewed Aerial Vehicles): Für Anwendungen in der Logistik und Überwachung liegt der TRL oft bereits im Bereich 7 bis 9. Viele Systeme befinden sich bereits im realen Testeinsatz oder in spezialisierten kommerziellen Nischen, wie der in den Quellen erwähnten medizinischen Logistik. eVTOL (Urban Air Mobility): Diese Technologie befindet sich für den Passagiertransport größtenteils in der Prototypen- und Zertifizierungsphase, was einem TRL von etwa 6 bis 8 entspricht. Die Quellen weisen darauf hin, dass das Interesse an diesen Operationen aufgrund technischer Fortschritte bei Elektroantrieben und digitalem Flugverkehrsmanagement stark gestiegen ist.

In [38]:
```
# ============================================================
# AEROVALE Market-Share Scenario Generator (1000+) + Plot
# - Realistic assumptions:
#   UAV entry:   2026–2032, K ~ 0.18 ± 0.05
#   eVTOL entry: 2028–2035, K ~ 0.12 ± 0.03
# - Adoption: logistic S-curve (diffusion)
# - Shares: always >=0 and sum to 1 (Dirichlet jitter around expected sha
# ============================================================

def logistic(t, K, r, tau):
    return K / (1.0 + np.exp(-r * (t - tau)))

def truncnorm_sample(rng, mean, std, low, high):
```

```
    # simple rejection sampler (fast enough for scalar draws)
    while True:
        x = rng.normal(mean, std)
        if low <= x <= high:
            return float(x)

def fan_stats(x):  # x: (n_scen, T)
    q10 = np.quantile(x, 0.10, axis=0)
    q50 = np.quantile(x, 0.50, axis=0)
    q90 = np.quantile(x, 0.90, axis=0)
    return q10, q50, q90

def plot_share_fan(ax, years, share_paths, title, n_example=30, rng=None)
    if rng is None:
        rng = np.random.default_rng(0)

    q10, q50, q90 = fan_stats(share_paths)

    # example paths
    n_scen = share_paths.shape[0]
    idx = rng.choice(n_scen, size=min(n_example, n_scen), replace=False)
    ax.plot(years, share_paths[idx].T, linewidth=0.8, alpha=0.12)

    # fan + median
    ax.fill_between(years, q10, q90, alpha=0.25, label="P10–P90")
    ax.plot(years, q50, linewidth=2.0, label="Median (P50)")

    ax.set_title(title)
    ax.set_xlabel("Year")
    ax.set_ylabel("Market share")
    ax.set_ylim(0, 1)

def generate_market_share_scenarios_realistic(
    years,
    n_scen=1000,
    groups=("conv", "uav", "evtol"),
    # Entry windows (calendar years)
    entry_uav=(2026, 2032),
    entry_evtol=(2028, 2035),
    # Saturation K (mean/std) = your "realistic values"
    K_uav_mean=0.18, K_uav_std=0.05, K_uav_bounds=(0.0, 0.50),
    K_evt_mean=0.12, K_evt_std=0.03, K_evt_bounds=(0.0, 0.40),
    # Adoption speed r (positive). These are “reasonable defaults”; tune
    r_uav_logn=(np.log(0.60), 0.40),   # mean in log-space, sigma
    r_evt_logn=(np.log(0.50), 0.50),
    # Time from entry to "take-off" midpoint (years)
    takeoff_delay=(2, 8),
    # Dirichlet concentration: higher = smoother/less noisy around the S-
    kappa=200,
    # reproducibility
    seed=42,
):
    rng = np.random.default_rng(seed)

    T = len(years)
    G = len(groups)
    idx = {g: i for i, g in enumerate(groups)}
    t = np.arange(T)

    shares = np.zeros((n_scen, T, G), dtype=float)
```

```
    for s in range(n_scen):
        # --- entry years -> indices ---
        y0_uav = int(rng.integers(entry_uav[0], entry_uav[1] + 1))
        y0_evt = int(rng.integers(entry_evtol[0], entry_evtol[1] + 1))

        t0_uav = int(np.clip(y0_uav - years[0], 0, T - 1))
        t0_evt = int(np.clip(y0_evt - years[0], 0, T - 1))

        # --- saturation levels K (truncated normal around your mean/std)
        Ku = truncnorm_sample(rng, K_uav_mean, K_uav_std, K_uav_bounds[0]
        Ke = truncnorm_sample(rng, K_evt_mean, K_evt_std, K_evt_bounds[0]

        # --- growth steepness r (lognormal, positive) ---
        ru = float(rng.lognormal(mean=r_uav_logn[0], sigma=r_uav_logn[1])
        re = float(rng.lognormal(mean=r_evt_logn[0], sigma=r_evt_logn[1])

        # --- inflection (midpoint) tau = entry + delay ---
        du = int(rng.integers(takeoff_delay[0], takeoff_delay[1] + 1))
        de = int(rng.integers(takeoff_delay[0], takeoff_delay[1] + 1))
        tau_u = t0_uav + du
        tau_e = t0_evt + de

        # --- raw penetrations (expected shares before normalization/jitt
        pu = logistic(t, Ku, ru, tau_u)
        pe = logistic(t, Ke, re, tau_e)
        pu[:t0_uav] = 0.0
        pe[:t0_evt] = 0.0

        # --- ensure feasibility: conv is the remainder; if pu+pe > 1, sc
        new_sum = pu + pe
        scale = np.ones(T)
        mask = new_sum > 1.0
        scale[mask] = 1.0 / new_sum[mask]
        pu2 = pu * scale
        pe2 = pe * scale
        conv = 1.0 - (pu2 + pe2)

        # expected share vector per year
        m = np.vstack([conv, pu2, pe2]).T  # (T,3)

        # --- Dirichlet jitter to create realistic year-to-year variabili
        alpha = np.clip(kappa * m, 1e-9, None)
        w = np.array([rng.dirichlet(alpha[t_i]) for t_i in range(T)])  #

        shares[s, :, idx["conv"]] = w[:, 0]
        shares[s, :, idx["uav"]]  = w[:, 1]
        shares[s, :, idx["evtol"]] = w[:, 2]

    return shares

# ========================
# RUN (edit these)
# ========================
start_year = 2025
horizon_years = 20  # same planning horizon as Task 2
years = np.arange(start_year, start_year + horizon_years + 1)  # inclusiv

n_scen = 1000
shares = generate_market_share_scenarios_realistic(years, n_scen=n_scen)
```

```
# ==========================
# PLOT (subplots)
# ==========================
groups = ("conv", "uav", "evtol")
idx = {g: i for i, g in enumerate(groups)}
rng_plot = np.random.default_rng(123)

uav  = shares[:, :, idx["uav"]]
evtl = shares[:, :, idx["evtol"]]
conv = shares[:, :, idx["conv"]]
new  = uav + evtl

fig, axes = plt.subplots(2, 2, figsize=(14, 8), sharex=True)

plot_share_fan(axes[0, 0], years, uav,  "UAV market share scenarios (1000
plot_share_fan(axes[0, 1], years, evtl, "eVTOL market share scenarios (10
plot_share_fan(axes[1, 0], years, conv, "Conventional share scenarios (10
plot_share_fan(axes[1, 1], years, new,  "New mobility share (UAV + eVTOL)

# shared legend
handles, labels = axes[0, 0].get_legend_handles_labels()
fig.legend(handles, labels, loc="lower center", ncol=2, frameon=False)

plt.tight_layout(rect=(0, 0.05, 1, 1))
plt.show()

# ==========================
# (Optional) quick sanity checks
# ==========================
# print("Min share:", shares.min(), "Max share:", shares.max())
# print("Sum close to 1 (max abs error):", np.max(np.abs(shares.sum(axis=
```



4. Options in and on systems

4.1 Options on Systems:

- Option to Defer (Vertiports): Postpone the construction of Vertiports for eVTOLS until demand is sufficient. Keeping open space to implement
- Option to Stage (UAV Cargo Ports): Starting with a small Cargo UAV Terminal for example within GA Section helps test out with the option to gradually grow

4.2 Options in Systems:

- Rooftop Vertiports: When a new Parking lot or other Terminal or Landside buildings are created, build them with the option to include Vertiports on the rooftop in the future as they provide a valuable and space-saving option.
- Build for Electricity: When building Apron and other new ground infrastructure, already include "leerrohre" for future electrification and recharging options for electric Aircraft.

5. Candidate Flexibility

Start with eVTOL Vertipads

At the beginning stage, we only want to create 1 or 2 trial eVTOL Vertipads with a charging station at a place where we can efficiently leave the airport terminal airspace. But it will be installed at a location, where we are able to expand the number of Vertipads without touching already existing infrastructure (option on system). Further, we want to already build the charging infrastructure and cabling in a way that we can expand the number of charging stations later without the need to completely rebuild the electric supply.